

ECE/CS 552 Introduction to Computer Architecture
Fall 2019
University of Wisconsin-Madison
Prof. Joshua San Miguel

Midterm Exam
Tuesday, October 22, 2019
75 Minutes, In-Class

1. The exam is two-sided and has 14 pages, which includes one blank page, one page of additional pipeline diagrams and the two-page MIPS Green Card in the end.
2. Closed book/closed notes examination.
3. No handheld computers or portable computers allowed.
4. Calculator allowed.
5. A single 8.5"x11" double-sided, hand-written cheat sheet is allowed.
6. **You must show your complete work.** Points will be awarded only based on what appears in your answers.
7. No one should leave the room during the last 5 minutes of the examination.
8. Upon announcement of the end of the exam, stop writing on the exam paper immediately. Wait for proctors to pick up all exams. The instructor will announce when you may leave the room.
9. Failure to follow instructions may result in forfeiture of your exam and will be handled according to UWS 14 academic misconduct procedures.

FIRST (GIVEN) NAME: _____

LAST (FAMILY) NAME: _____ **SOLUTION** _____

STUDENT ID#: _____

This page is left intentionally blank, in case you need extra space for answers.

	Achieved Marks	Maximum Marks
Problem 1		/ 10
Problem 2		/ 10
Problem 3		/ 12
Problem 4		/ 10
Problem 5		/ 18
Problem 6		/ 8
Total		/ 68

Problem 1 (10 marks)

Consider the following MIPS assembly program:

```
I1: sw    $s1, 0($s3)
I2: add   $s3, $s1, $s0
I3: lw    $s2, 4($s3)
I4: lw    $s3, 0($s2)
I5: beq   $s3, $s2, TARGET
```

- (a) (8 marks) In the table below, list all the **register data dependences** (RAW, WAW, WAR). Specify the register as well as the older and younger instructions involved.

Register	Older Instruction	Younger Instruction	Type of Dependence
\$s3	I2	I3	RAW
\$s2	I3	I4	RAW
\$s2	I3	I5	RAW
\$s3	I4	I5	RAW
\$s3	I2	I4	WAW
\$s3	I1	I2	WAR
\$s3	I3	I4	WAR

- (b) (2 marks) In no more than two sentences, explain why **WAR register data dependences** do not cause hazards in the five-stage pipeline (IF-ID-EX-MEM-WB) from class.

Since 1) every instruction reads its source registers (ID) before writing to its destination register (WB), and 2) all instructions read/write registers in the order specified by the program, it is not possible for a younger instruction to write to a register before it is read by an older instruction.

Problem 2 (10 marks)

You build a simple **single-cycle processor** to run your graph analytics program. You decide to implement a new combined-store instruction that is able to replace **every two store instructions** in your program with a single combined-store instruction. When running the new version of your program, you observe that it now takes **1.2x less cycles** to execute.

- (a) (4 marks) What fraction of instructions in your original graph analytics program were store instructions?

Let x be the fraction of stores in the original program:

$$1.2 = 1 / (1 - x + x/2)$$

Thus $x = 1/3$

- (b) (6 marks) On top of this, you also decide to implement a new combined-load instruction that is able to replace **every two load instructions** with a single combined-load instruction. However, this increases the critical path of your processor and thus **increases the clock period by 1.5x**. After some experiments, you find that enabling both the combined-store and combined-load instructions simultaneously now actually yields the **same total execution time** as with the original version of your graph analytics program (without either combined-store nor combined-load instructions).

What fraction of instructions in your original program (without either combined-store nor combined-load instructions) were load instructions?

If total speedup is 1x, and clock period increases by 1.5x, then the number of cycles must decrease by 1.5x.

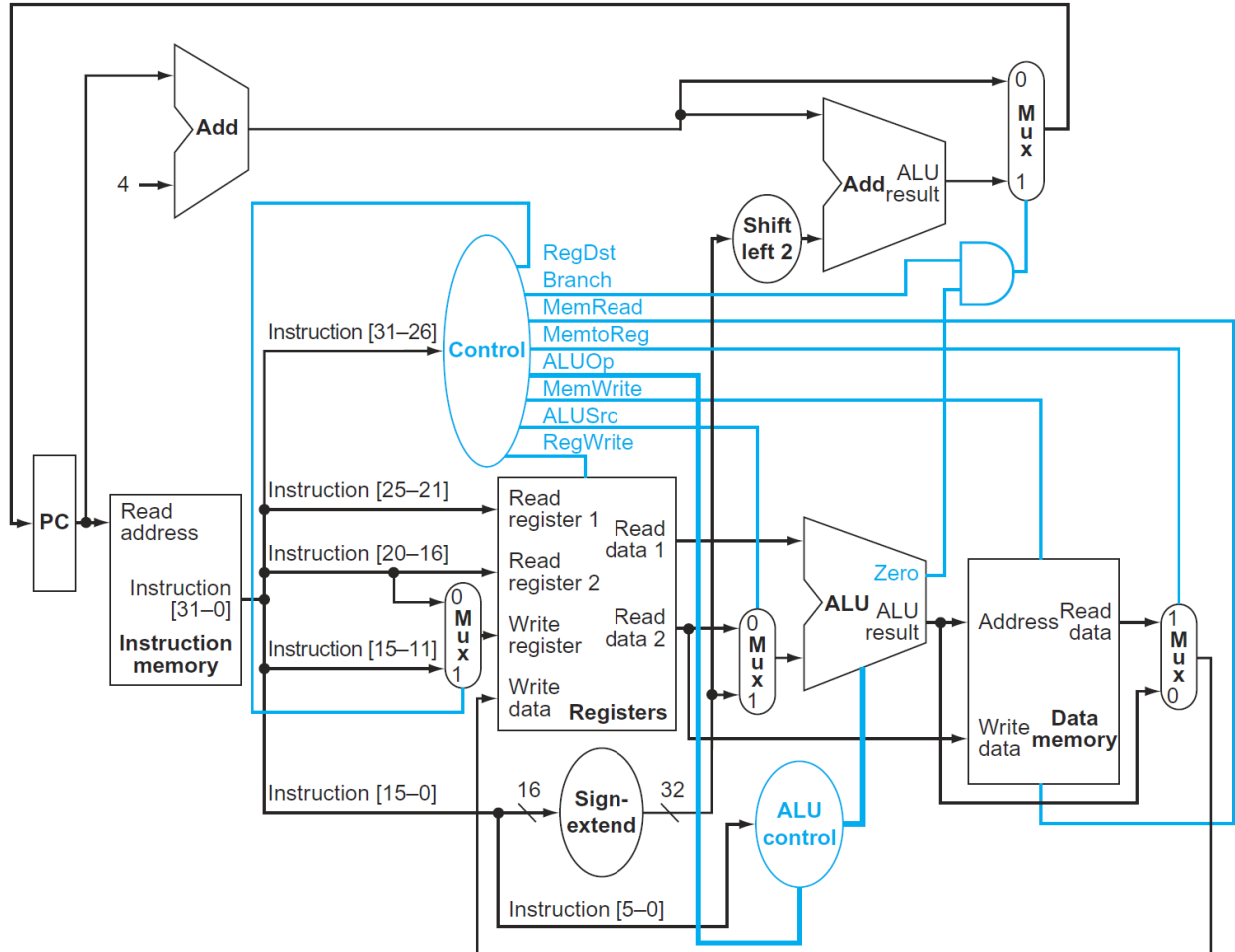
Let y be the fraction of loads in the original program:

$$1.5 = 1 / (1 - x - y + x/2 + y/2), \text{ where } x = 1/3 \text{ as above}$$

Thus $y = 1/3$

Problem 3 (12 marks)

You build the simple single-cycle MIPS processor from class, shown below.



- (a) (2 marks) Give an example instruction that would not execute correctly if you were to enable **register file bypassing** in your single-cycle processor. Explain why, in no more than two sentences.

e.g., `add $t0, $t0, $t1`

This instruction would fail since it is overwriting the same register that it is reading. With RF bypassing enabled, this instruction would end up reading the value that it is writing, as opposed to the current value of [\$t0].

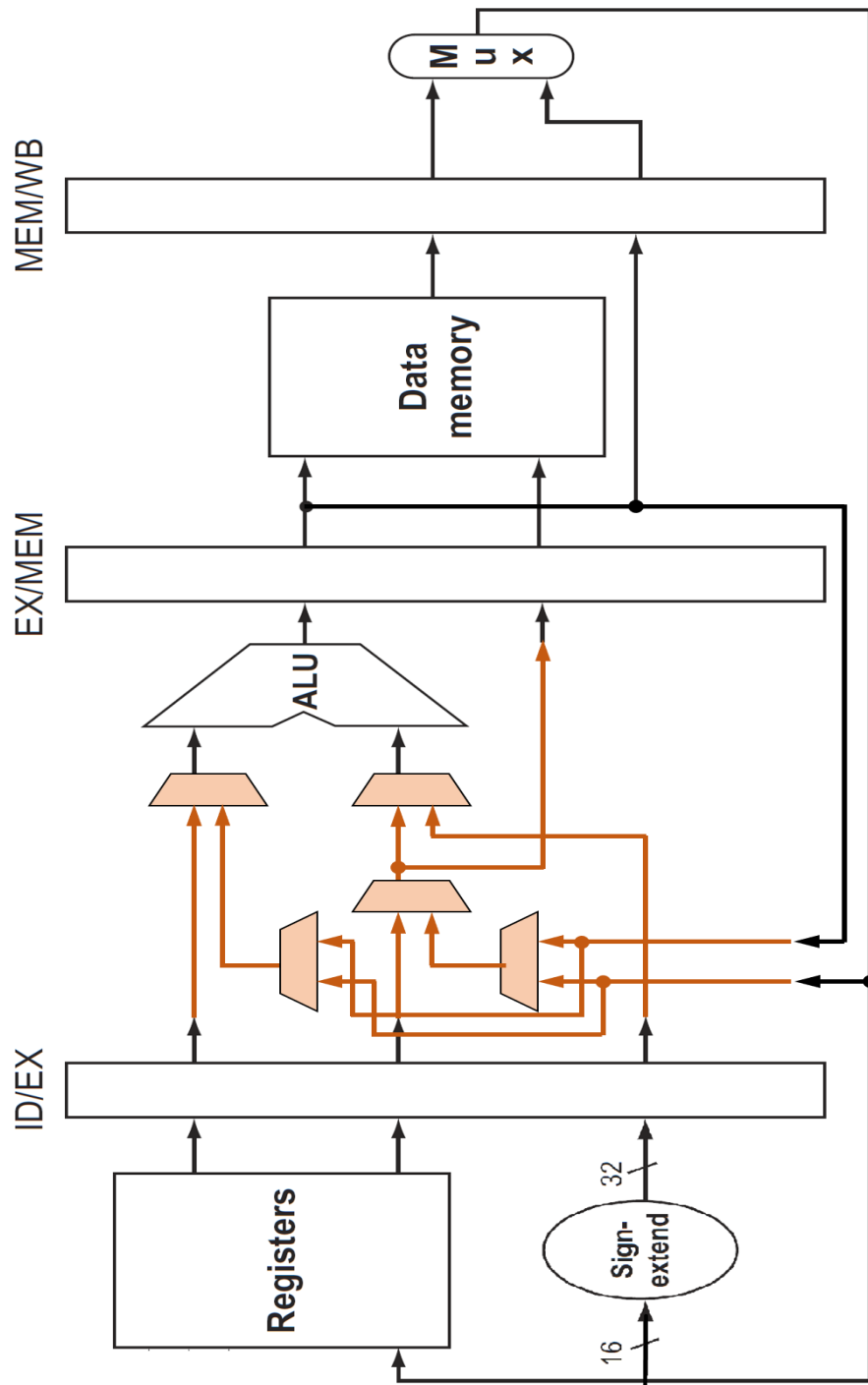
- (b) (10 marks) Assume that the control logic in your processor is buggy and **always incorrectly assigns both of the following**:
- $\text{RegDst} = \text{RegWrite} \ \& \ (\text{MemWrite} \mid \text{Instruction}[28] \mid \sim \text{MemtoReg})$
 - $\text{ALUSrc} = (\text{Instruction}[25] \mid \text{MemRead}) \ \& \ \sim \text{Branch}$

For each instruction below, specify whether or not it would still execute correctly on this buggy processor.

Instruction	Executes Correctly? Yes or No
<code>sub \$t0, \$s0, \$t1</code>	No, ALUSrc incorrect
<code>beq \$t3, \$zero, LOOP</code>	Yes
<code>lw \$s6, 8(\$t2)</code>	Yes
<code>addi \$s1, \$s1, 1</code>	No, RegDst incorrect
<code>sw \$s4, 4(\$t2)</code>	No, ALUSrc incorrect

Problem 4 (10 marks)

You are tasked with implementing the datapath for **EX-to-EX** and **MEM-to-EX forwarding** in a five-stage pipeline (IF-ID-EX-MEM-WB) processor from class. Unfortunately, for budget reasons, you only have 2-to-1 multiplexers available. Complete the datapath of the EX stage in the diagram below using **only wires and 2-to-1 multiplexers**. Ignore branch, shift and jump instructions. You do not need to draw the select signals for the multiplexers.



Problem 5 (18 marks)

You build a processor that uses the five-stage pipeline (IF-ID-EX-MEM-WB) from class. Assume that the following are all enabled: **RF bypassing**, **EX-to-EX forwarding**, **MEM-to-EX forwarding** and **MEM-to-MEM forwarding**. Assume that branches are resolved in the ID stage of the pipeline. Use a **predict-not-taken** policy with branch delay slots disabled. Assume that forwarding is available to the branch resolution circuit at the ID stage. Consider the following MIPS program running on your processor:

```

I1:      sw $t0, 8($t1)
I2:      beq $t0, $t3, I4
I3:      addi $t2, $t2, -9
I4:      lw $t0, 8($t1)
I5:      sub $t3, $t2, $t3
I6:      sw $t1, 4($t3)

```

Just before executing this code, the register contents are as follows (in signed decimal):

\$t0	\$t1	\$t2	\$t3
16	7000	45	20

(a) (6 marks) Complete the pipeline timing diagram below up until I6 reaches the WB stage.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
I1	F	D	X	M	W														
I2		F	D	X	M	W													
I3			F	D	X	M	W												
I4				F	D	X	M	W											
I5					F	D	X	M	W										
I6						F	D	X	M	W									

(b) (2 marks) Specify at which cycles (if any) EX-to-EX forwarding is enabled.

Cycle 8

(c) (2 marks) Specify at which cycles (if any) MEM-to-EX forwarding is enabled.

Cycle 7

(d) (8 marks) Assume that a random particle strike during **cycle 7** causes **bit 0** of the control signal **ID/EX.RegisterRs** to get **stuck at 1**. This only affects cycle 7; subsequent cycles are back to normal. Fill in the register contents below after I6 completes the WB stage. Marks are not evenly distributed; some registers are worth more marks than others.

Register	Contents (in signed decimal)
\$t0	16
\$t1	7000
\$t2	36
\$t3	25

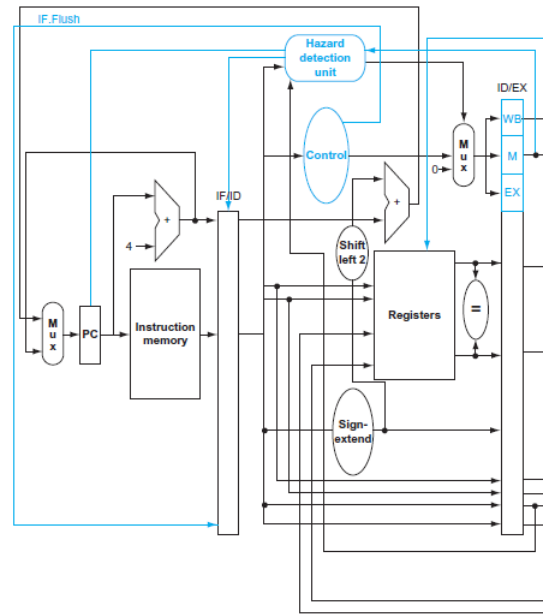
During cycle 7, the particle strike disables the MEM-to-EX forwarding path from I3 to I5; thus I5 ends up reading the stale value of [\$t2] (= 45) that is currently in the register file. Note that though ID/EX.RegisterRs would be equal to \$t3 due to this particle strike, I5 would not read the contents of [\$t3] because the instruction is now past the ID stage.

Problem 6 (8 marks)

You build a processor that uses the five-stage pipeline (IF-ID-EX-MEM-WB) from class.

- (a) (3 marks) The figure on the right shows an implementation where branches are predicted to be not taken and are resolved in the ID stage. Describe what hardware changes would be necessary to implement a **predict-taken** policy instead.

Branch target computation now needs to be performed in the IF stage. Specifically, the components needed for computing the branch target (sign-extend, shift left 2, branch target adder) need to be moved to IF, as well as their corresponding control logic.



- (b) (5 marks) Assume that you implement support for register file bypassing **but not EX-to-EX, MEM-to-EX nor MEM-to-MEM forwarding**; instead your processor stalls for the minimum number of cycles to resolve data hazards. Consider the following MIPS program running on your processor. Fill in the blanks such that the program takes **exactly 23 cycles** to execute.

```

or    $s5, $s2, $s3
add   $s2, $s2, $s3
slt   $s1, $s5, $zero
lw    $s2, 0($s1)
lw    $s3, 4($s2)
add   $s4, $s3, $s3
sw    $s2, 0($s5)
and   $s5, $s3, $s4
sw    $s5, 0($s1)

```

This page provides additional pipeline diagrams, in case you need them.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19